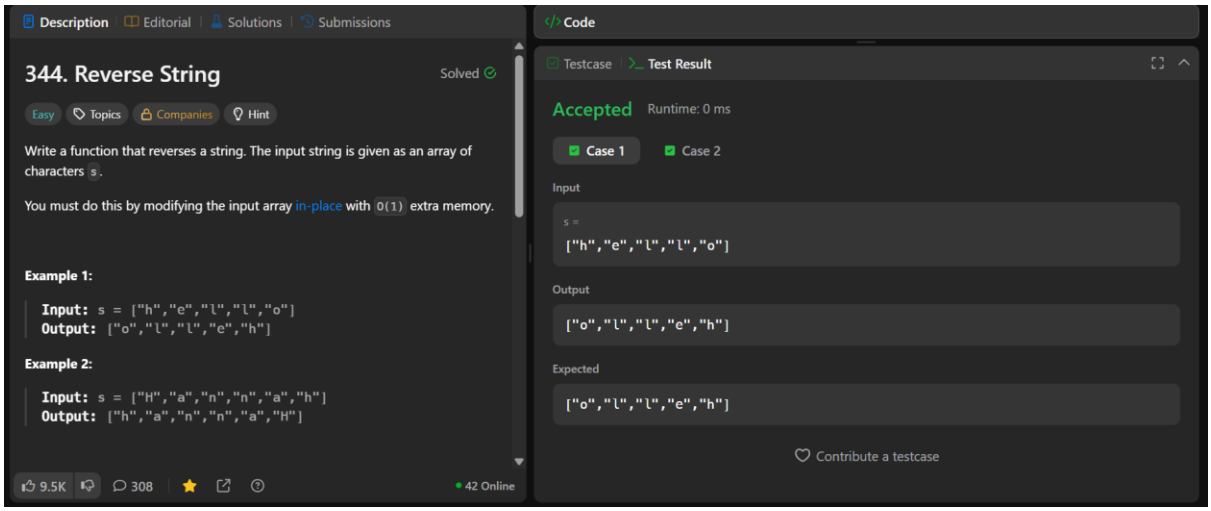
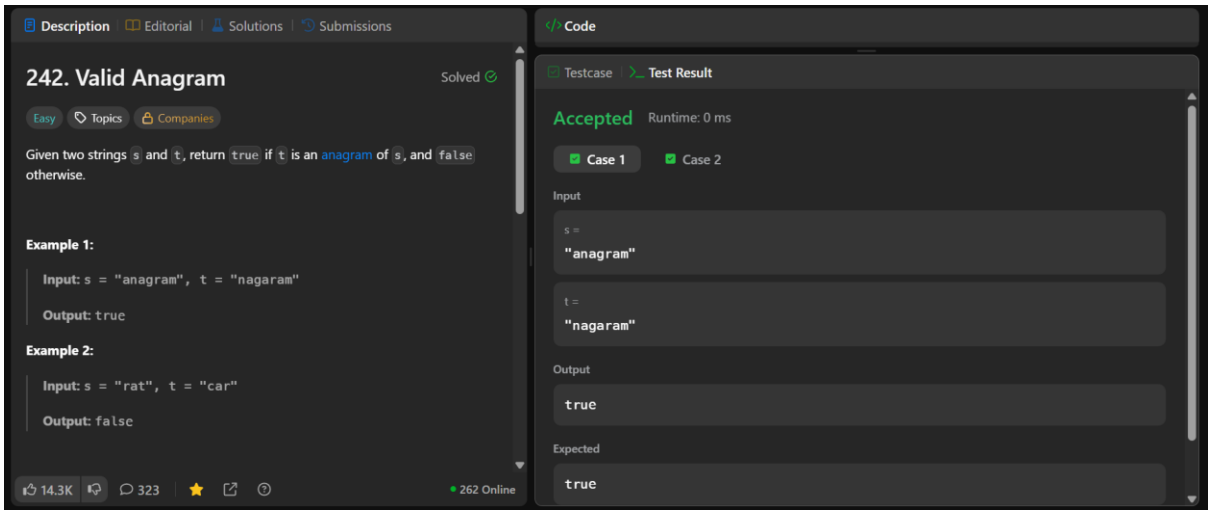
```
class Solution {
public:
    void reverseString(vector<char>& s) {
        int l=0;
        int r=s.size()-1;
        while(l<r){
            swap(s[l],s[r]);
            l++;
            r--;
        }
    }
};
```



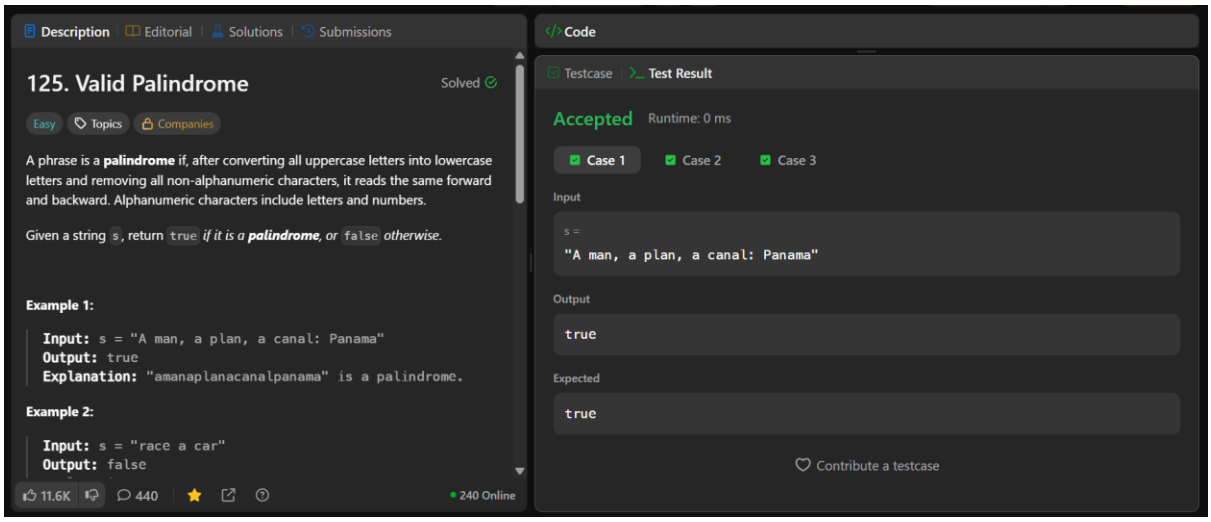

```
class Solution {
public:
    bool isAnagram(string s, string t) {
        if (s.length()!=t.length()){
            return false;
        }
        sort(s.begin(), s.end());
        sort(t.begin(), t.end());
        return (s == t);
    }
```

};

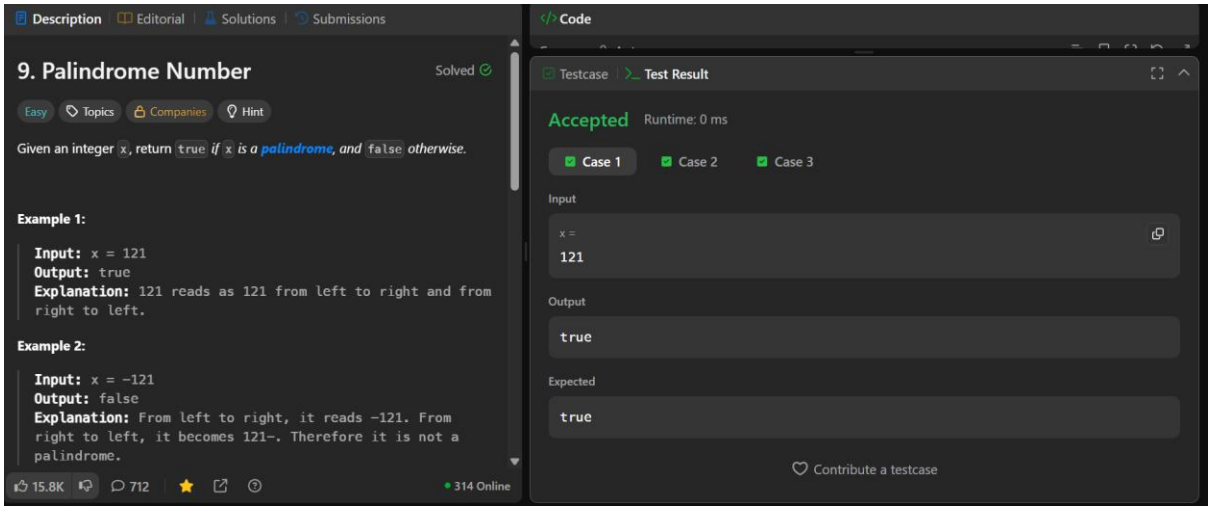



```
class Solution {
public:
 string cleanString(string s) {
   string result = "";

   for (int i = 0; i < s.length(); i++) {
      char c = s[i];

      if (isalnum((unsigned char)c)) {
         result += tolower((unsigned char)c);
      }
   }
   return result;
}

   bool isPalindrome(string s) {
      s = cleanString(s);

      string rev = string(s.rbegin(), s.rend());
      return s == rev;
   }
```

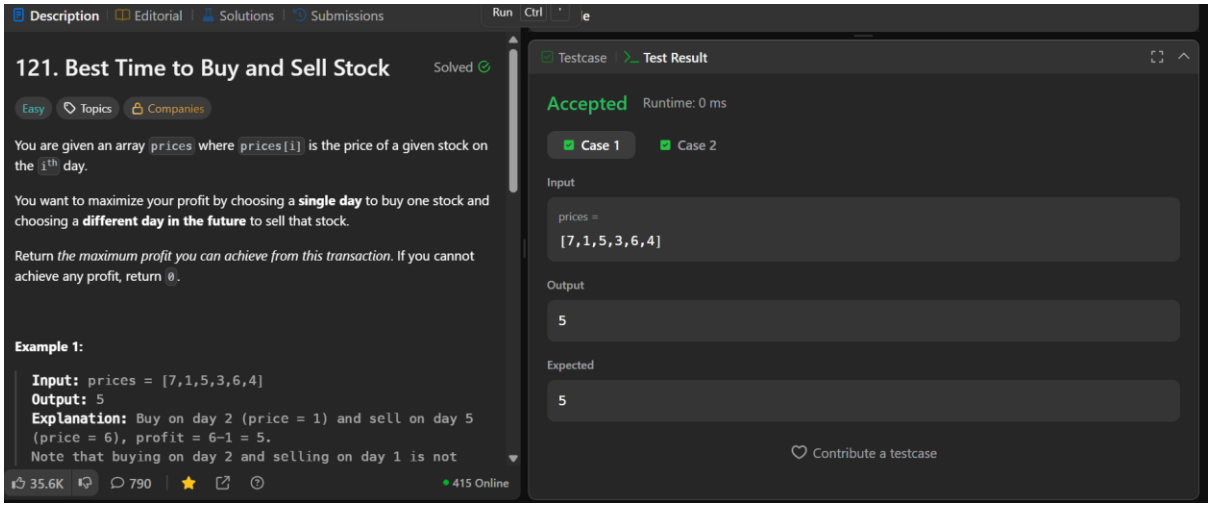
};



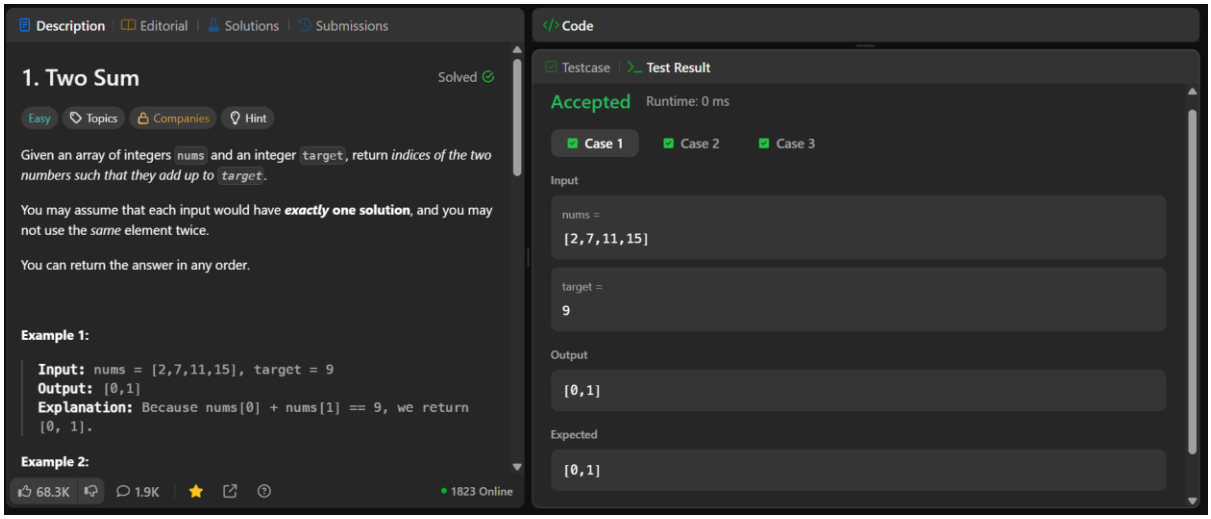

```
class Solution {
public:
int reverseDigits(int num) {
    long long rev_num = 0;
    while (num > 0) {
        rev_num = rev_num * 10 + num % 10;
        num = num / 10;
  }
  return rev_num;
}
bool isPalindrome(int n) {
  int rev_n = reverseDigits(n);
  return (rev_n == n);
}

};
```




```
class Solution {
public:
   int maxProfit(vector<int>& prices) {
      int n=prices.size();
      int res=0;
      int m = prices[0];
      for(int i=1;i<n;i++){
            res = max(res, prices[i]- m);
            m=min(m,prices[i]);
         }
         return res;
      }

};
```

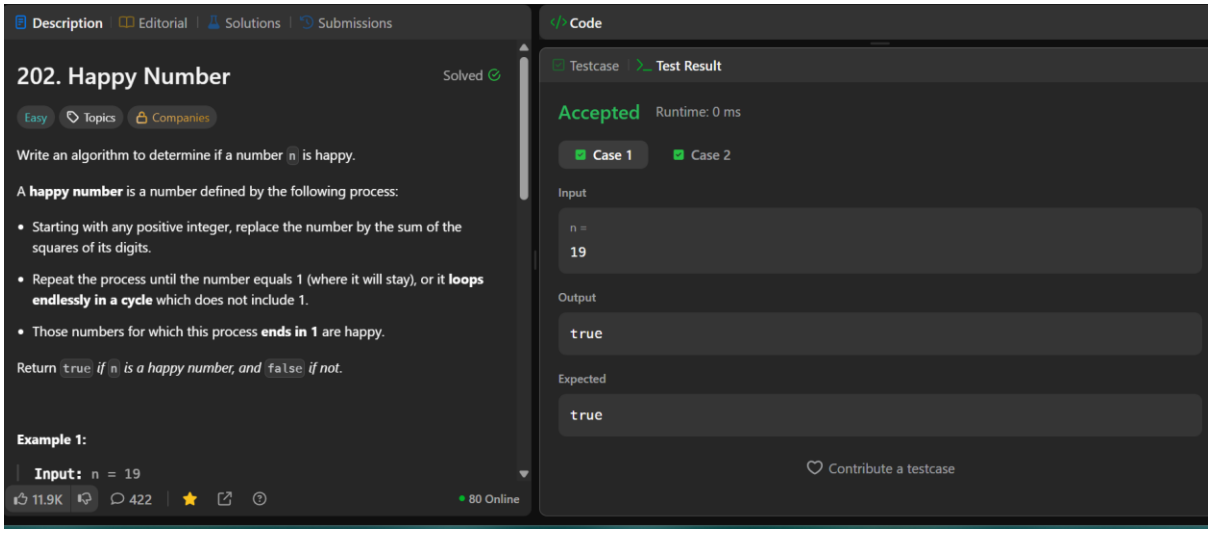



```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        int n = nums.size();
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (nums[i] + nums[j] == target) {
                    return {i, j};
                }
            }
        }
        return {};
    }
};
```




```
class Solution {
public:
    int numSquareSum(int n) {
        int sum = 0;
        while (n != 0) {
            int digit = n % 10;
            sum += digit * digit;
            n /= 10;
        }
        return sum;
    }

    bool isHappy(int n) {
        set<int> st;

        while (n != 1) {
            n = numSquareSum(n);

            if (st.find(n) != st.end())
                return false;

            st.insert(n);
```

```
        }
        return true;
    }
};
```


Description
Editorial
Solutions
Submissions
Code
202. Happy Number
Solved
Easy
Topics
Companies
Write an algorithm to determine if a number n is happy.
A happy number is a number defined by the following process:
Starting with any positive integer, replace the number by the sum of the squares of its digits.
Repeat the process until the number equals 1 (where it will stay), or it loops endlessly in a cycle which does not include 1.
Those numbers for which this process ends in 1 are happy.
Return true if n is a happy number, and false if not.
Example 1:
Input: n = 19
11.9K
422
80 Online
Testcase
Test Result
Accepted
Runtime: 0 ms
Case 1
Case 2
Input
n =
19
Output
true
Expected
true
Contribute a testcase